Control Flow 101

Guillermo Polito













Goal

- Understanding control flow representations
- Jumps and GOTOs
- Graphs

- Representing conditional and loop constructs

Changing the Execution Flow

- In a conditional statement some part of the code must ***not*** execute
- We need to model how to "skip" code

```
if (cond) {
  // True case
} else {
  // False case
}
```

Changing the Execution Flow

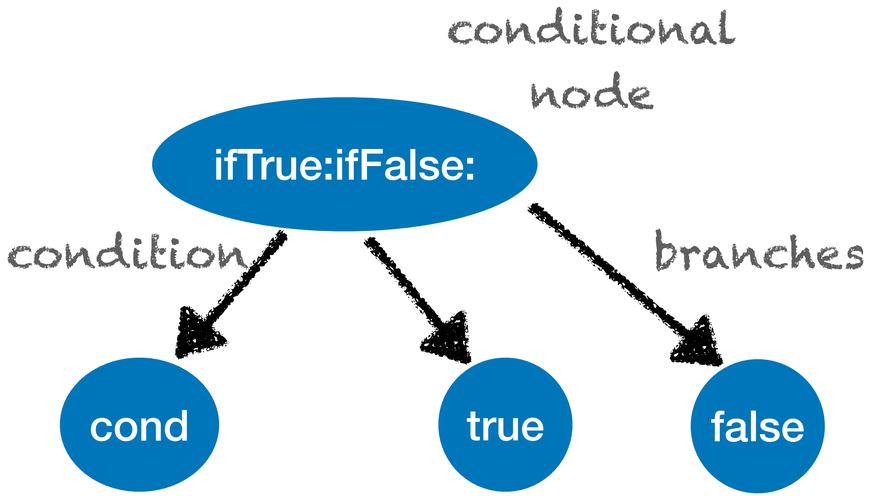
- In a conditional statement some part of the code must ***not*** execute
- We need to model how to "skip" code

```
if (cond) {
  // True case
} else {
  // False case
}
```

Tree Representation and Conditionals

- Trees represent branching naturally
- The conditional node decides
  - Does it execute one branch?
  - or the other?
- The catch: to implement it, we need to branch!

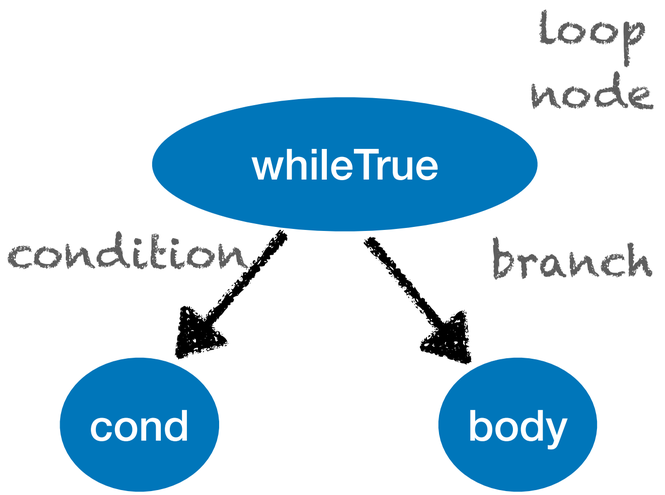
```
cond
  ifTrue: [ true ]
  ifFalse: [ false ]
```



Tree Representation and Loops

- Loops inherit the same simplicity
- The loop node decides
  - Does it execute the body?
  - or does it exit?
- The catch: to implement it, we need to branch!

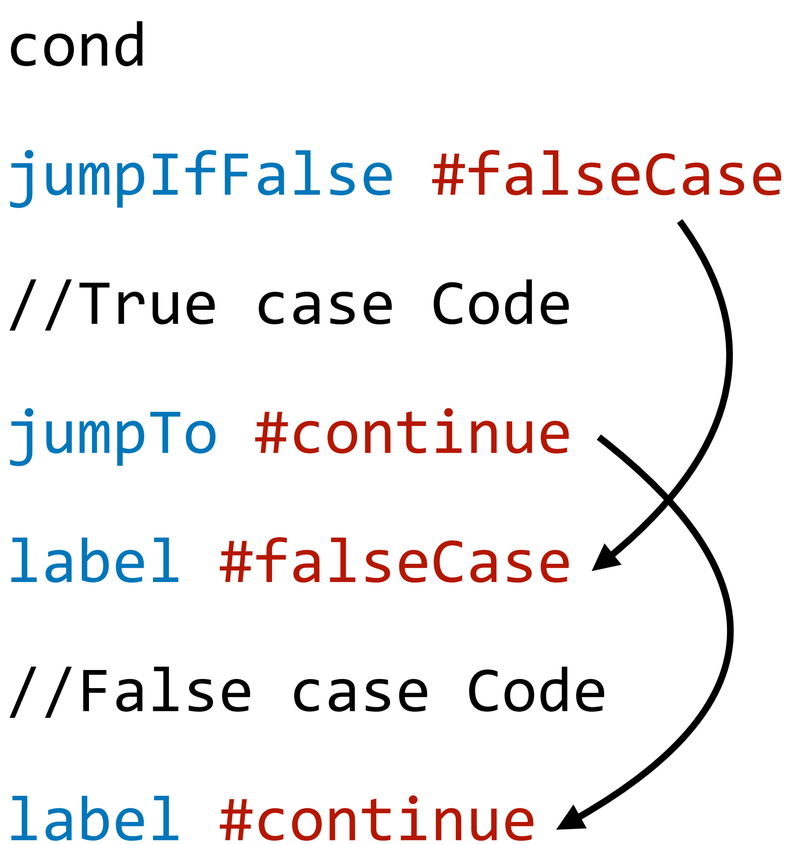
```
[ cond ] whileTrue: [ body ]
```



Linear Code and Conditionals

- Linear code needs to *jump*
- Conditionally and inconditionally
- Branches = Conditional jumps

```
if (cond) {
  // True case
} else {
  // False case
}
```

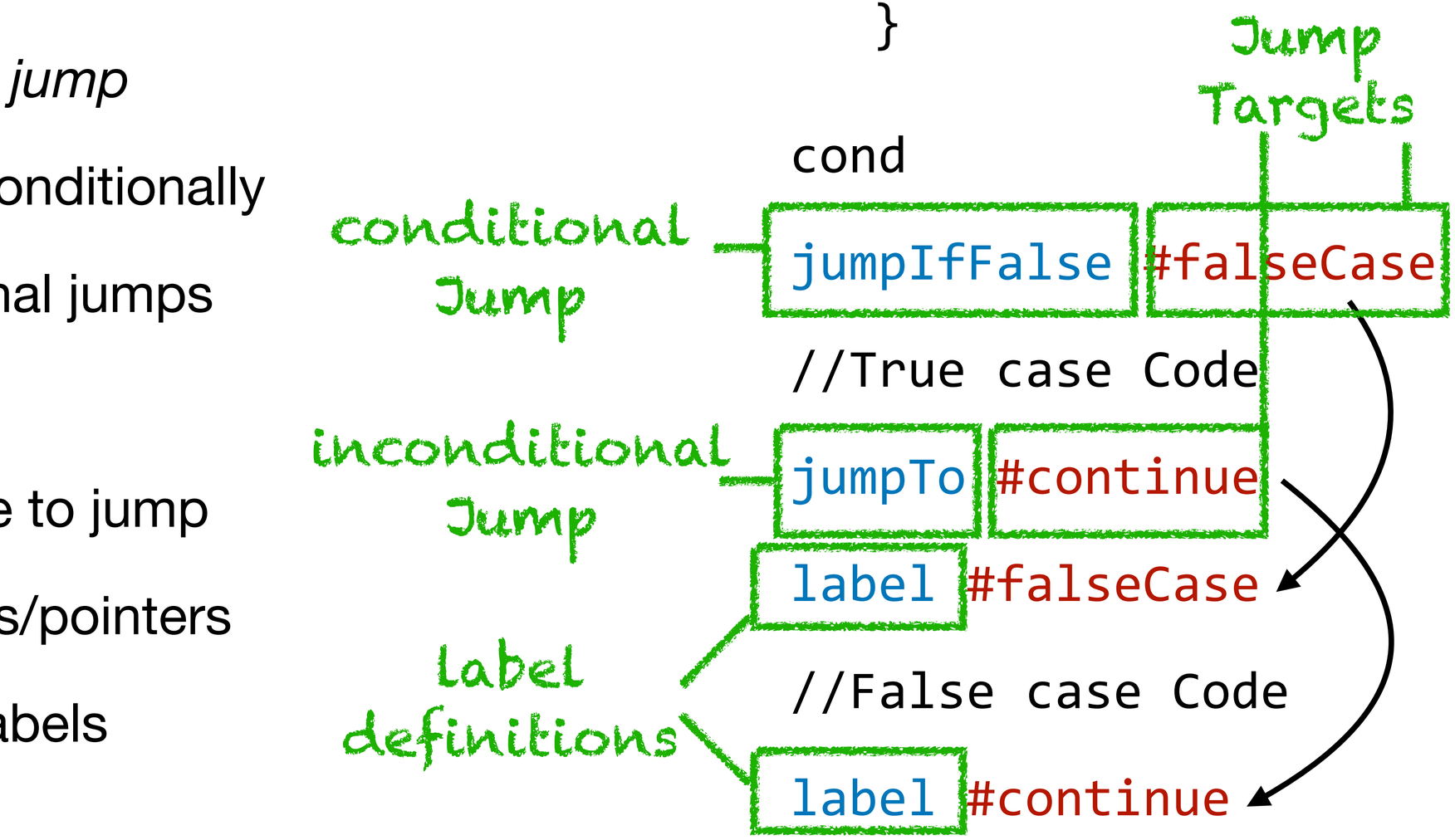


Linear Code and Conditionals

```
if (cond) {
  // True case
} else {
  // False case
}
```

- Linear code needs to *jump*
- Conditionally and inconditionally
- Branches = Conditional jumps

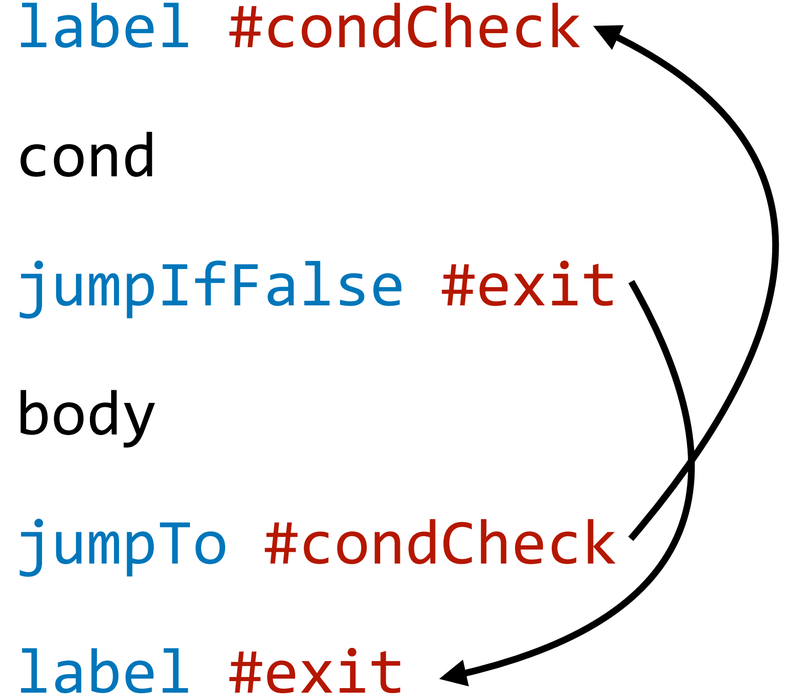
- Jump targets = where to jump
  - in reality: addresses/pointers
  - modeled through labels



Linear Code and Loops

```
[ cond ] whileTrue: [ body ]
```

- Loops need to jump *back*
- A.k.a., backjumps

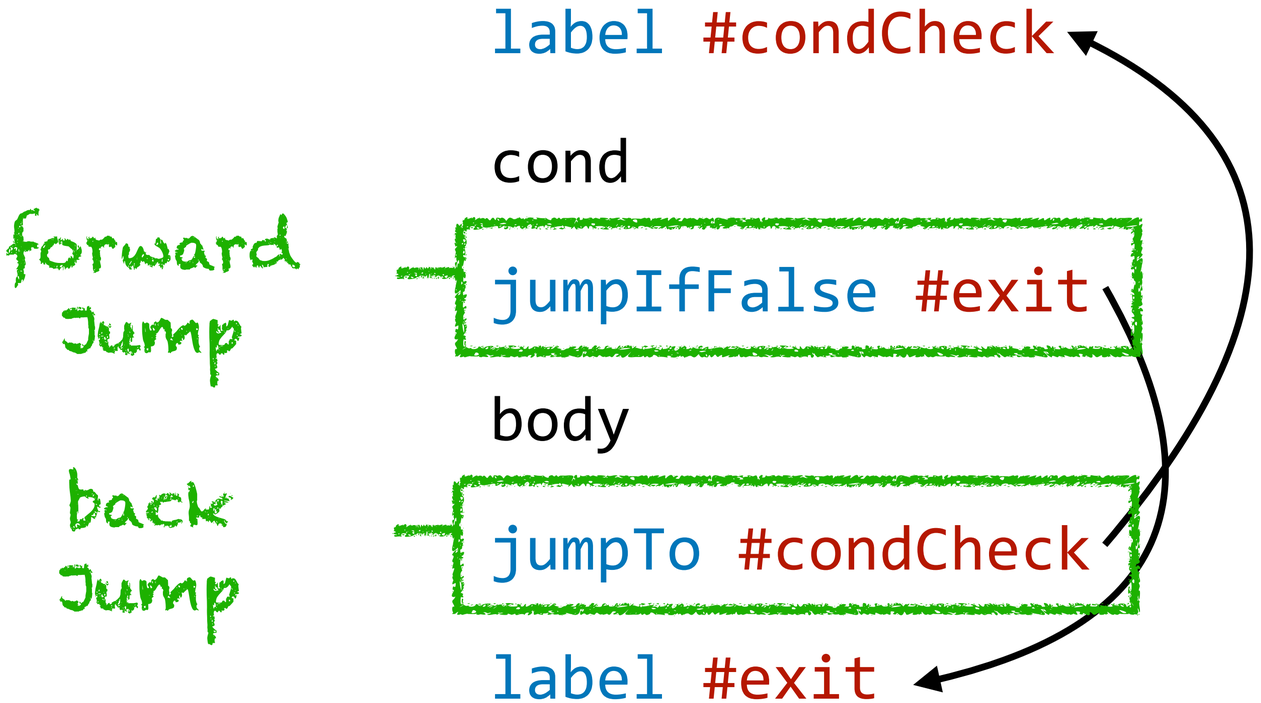


Linear Code and Loops

```
[ cond ] whileTrue: [ body ]
```

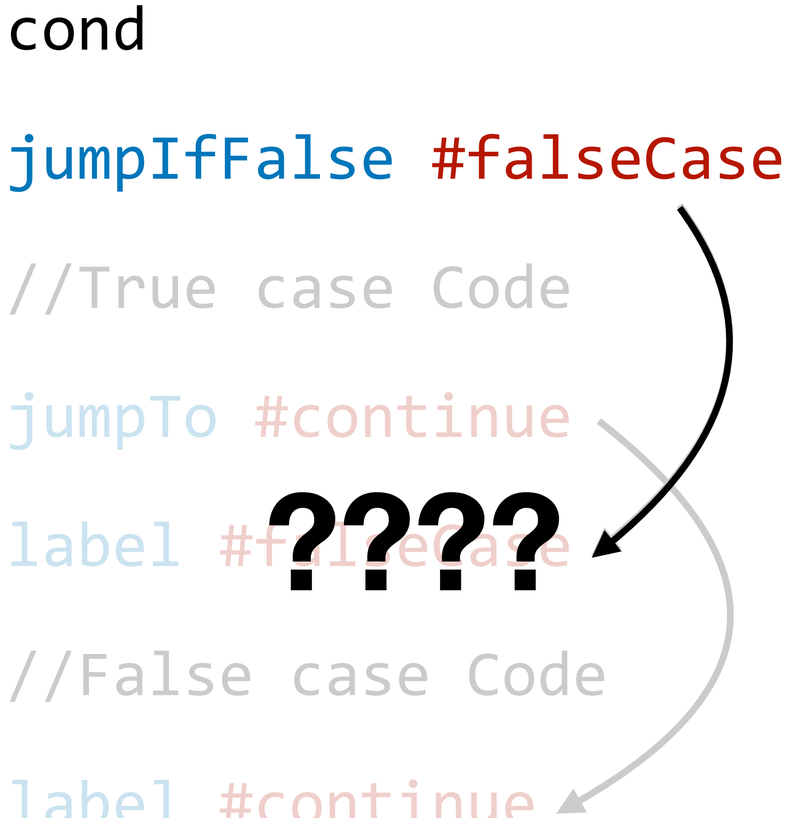
- Loops need to jump *back*
- A.k.a., backjumps

- Contrast with *forward* jumps



Producing Control Flow in Linear Code

- We must produce code in some order, usually top to bottom
- Thus
  - when jump instructions are created
  - the forward label instructions may not exist yet
- Common strategy: patching



Patching Control Flow

- Store the jump
- Set jump target when available

```
cond
theJump := jumpIfFalse ???
//True case Code
jumpTo #continue
theLabel := label #falseCase
theJump patchTarget: theLabel
//False case Code
label #continue
```

Takeaways

- Conditionals and Loops modify the control flow of the code
- Such control-flow modifications can be modeled in different ways
- Linear representations require most of the times GOTOs+labels

- Generating code may require strategies like patching

Material

- Books
  - Engineering a Compiler - IR design
  - Crafting Interpreters

